

INFORME · CORTE 2

Consola de monitoreo de red con alertas a Telegram

FortiGate, switch Huawei y router Huawei vigilados por SNMP desde un tablero web propio, desplegado con Docker.

Estudiantes

Ronald Toro - Daniel Steven Triviño

Repositorio

<https://github.com/RonaldT06/consola-monitoreo-toro-trivino>

Herramienta

NetMon (Python · FastAPI · SNMP · Docker)

Fecha

Octubre de 2026

1. Qué hace la herramienta

NetMon es una consola de monitoreo desarrollada desde cero en Python. Vigila tres equipos de red del laboratorio, muestra su estado en un tablero web en tiempo real y envía una alerta a Telegram cuando algo falla. No usa herramientas de terceros como Zabbix o Grafana: el sondeo SNMP, el motor de alertas, el tablero y el bot son código propio. Todo corre dentro de un contenedor Docker.

- **Sondeo SNMP cada 10 s:** uptime, CPU, memoria, temperatura, sesiones y estado y tráfico de cada interfaz.
- **Traps SNMP (UDP 162):** los equipos avisan por su cuenta cuando cae un enlace, y NetMon reacciona en segundos.
- **Tablero web** (puerto 8090): cada equipo en verde o rojo, gráficas de la última hora, alertas activas e historial.
- **Telegram:** mensaje inmediato ante caídas, interfaces desconectadas, reinicios o CPU/memoria altas, más los comandos `/estado` , `/interfaces` y `/alertas` .
- **Historial:** todos los eventos quedan guardados en una base SQLite, dentro de un volumen de Docker.

Alertas que genera

Evento	Cómo se detecta	Severidad
Equipo apagado o sin conexión	Fallan SNMP y ping en 2 sondeos seguidos	Crítica
Equipo recuperado (indica si se reinició)	Vuelve a responder; se compara el uptime	Informativa
Interfaz caída	<code>ifOperStatus</code> = down o trap linkDown	Crítica
Interfaz en shutdown	<code>ifAdminStatus</code> = down	Advertencia
CPU, memoria, temperatura o disco altos	Umbral superado en 2 sondeos seguidos	Advertencia
Responde ping pero no SNMP	Diagnóstico de community o ACL	Advertencia

2. Equipos monitoreados

Equipo	Rol	IP de gestión	Información obtenida (MIB)
FortiGate	Firewall	192.168.100.1	CPU, memoria, sesiones y disco (FORTINET-FORTIGATE-MIB); interfaces (IF-MIB)
Huawei S57xx	Switch	192.168.100.2	CPU, memoria y temperatura (HUAWEI-ENTITY-EXTENT-MIB); interfaces (IF-MIB)
Huawei AR	Router	192.168.100.3	CPU, memoria y temperatura (HUAWEI-ENTITY-EXTENT-MIB); interfaces (IF-MIB)
PC con Docker	Servidor NetMon	192.168.100.10	Ejecuta el contenedor; recibe traps en UDP 162

En cada equipo se habilitó SNMP v2c de **solo lectura**. El acceso está restringido a la IP del servidor: con una ACL en los Huawei y con la lista de `hosts` en el FortiGate. Los equipos envían sus traps a esa misma IP. Los comandos exactos están en la carpeta `configs/` del repositorio.

3. Cómo encaja todo

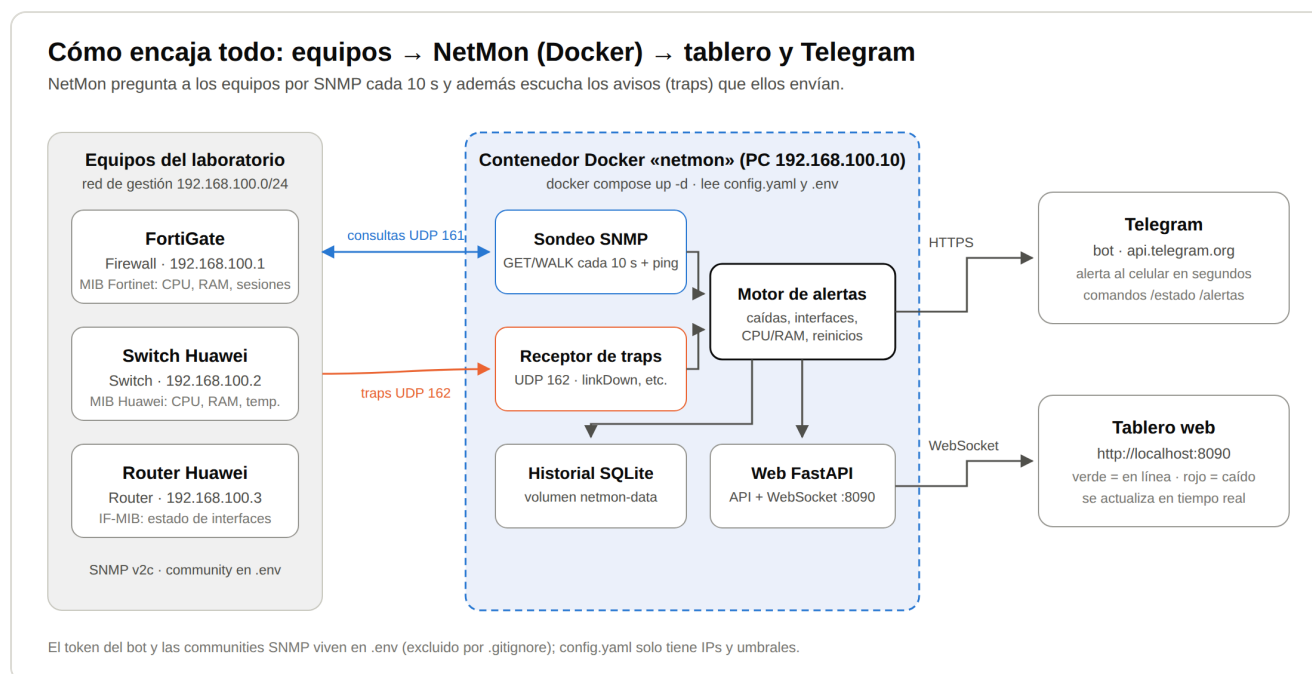


Figura 1. Los equipos responden consultas SNMP (UDP 161) y envían traps (UDP 162) al contenedor *netmon*, que alimenta el tablero web y el bot de Telegram.

1. El **sondeo SNMP** pregunta a cada equipo cada 10 s y entrega los datos al motor de alertas. Usa GET para valores únicos y WALK para tablas como las interfaces.
2. El **receptor de traps** escucha los avisos de los equipos. Un linkDown, por ejemplo, fuerza un sondeo inmediato, así la alerta llega en segundos.
3. El **motor de alertas** compara cada sondeo con el anterior y detecta cambios: equipo caído o recuperado, interfaz abajo o arriba, umbrales superados. Cada cambio se convierte en un evento.

4. Cada evento se guarda en **SQLite**, se envía a **Telegram** y se empuja al **tablero web** por WebSocket, sin necesidad de recargar la página.

Los secretos (token del bot, chat_id y communities SNMP) están en el archivo `.env`, que Docker inyecta al contenedor y que `.gitignore` excluye del repositorio. El archivo `config.yaml` solo contiene IPs y umbrales.

4. Pasos realizados

1 Configurar SNMP en los equipos

En cada equipo se aplicó la configuración de `configs/`, que hace tres cosas: activar SNMP v2c de solo lectura, permitir el acceso solo desde 192.168.100.10 y enviar los traps a esa misma IP. Por ejemplo, en el switch Huawei:

```
snmp-agent sys-info version v2c
snmp-agent community read cipher ***** acl 2099
snmp-agent target-host trap address udp-domain 192.168.100.10 params securityname
cipher ***** v2c
snmp-agent trap enable
```

2 Levantar la consola con Docker

Se copió `.env.example` como `.env`, se llenaron el token, el chat_id y las communities, y se levantó el contenedor con `docker compose up -d --build`. Para verificar que quedó corriendo, en estado *healthy* y publicando los puertos 8090/tcp y 162/udp, se usó `docker ps`.

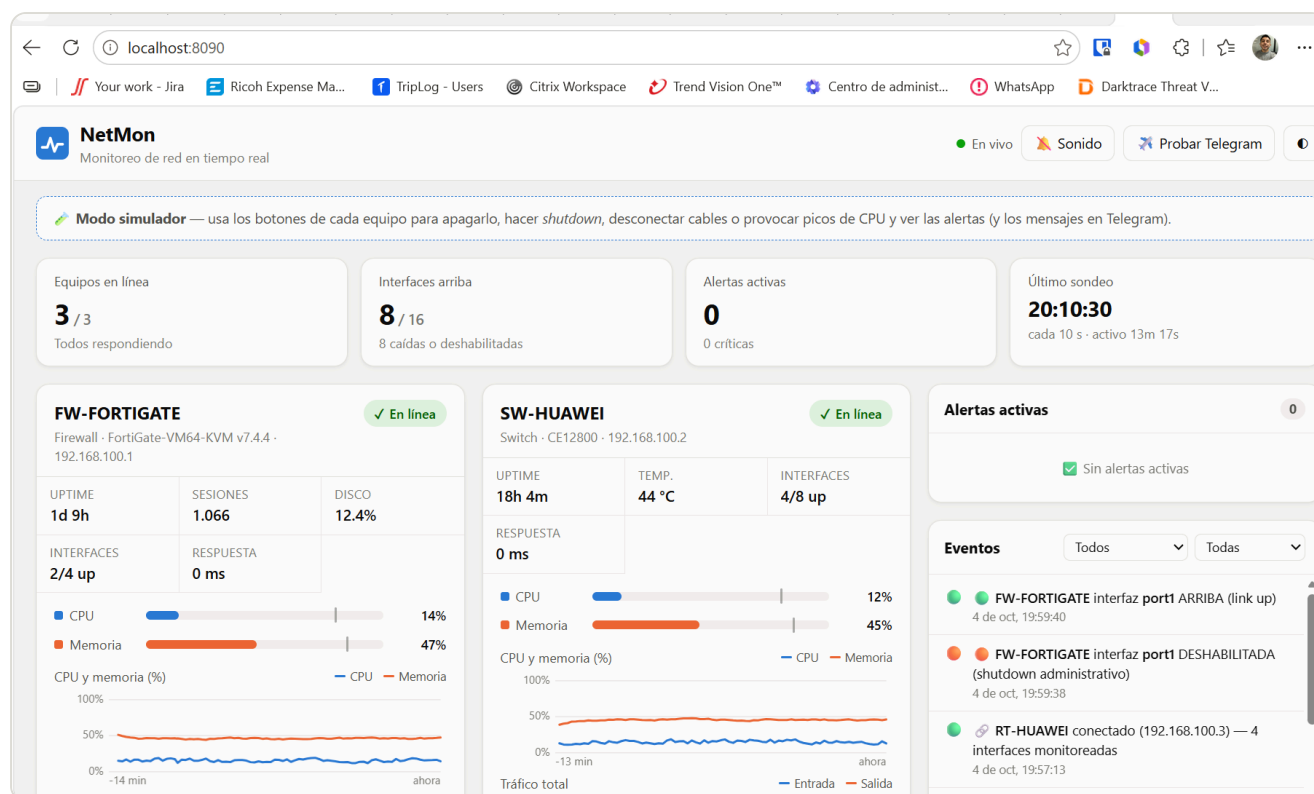
Nota sobre las pruebas: los equipos físicos del laboratorio no estuvieron disponibles al momento de las capturas. Por eso las pruebas 02–04 se hicieron con el **modo simulador** de NetMon (`NETMON_SIMULATE=1`), que reproduce los tres equipos con sus mismos nombres, IPs e interfaces. El contenedor, el tablero, el motor de alertas y el bot de Telegram son exactamente los mismos que con equipos reales. Para pasar a los equipos reales basta con aplicar la configuración de `configs/` y poner `NETMON_SIMULATE=0`.

```
PS C:\consola-monitoreo-toro> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS              PORTS
45ab7959a1e5   consola-monitoreo:latest            "python run.py --con..." 12 minutes ago Up 12 minutes (healthy) 0.0.0.0:162->162/udp, [::]:162->162/udp, 0.0.0.0:8090->8090/tcp, [::]:8090->8090/tcp netmon
PS C:\consola-monitoreo-toro>
```

Captura 01. El contenedor *netmon* en ejecución (`docker ps`).

3 Verificar el tablero con los equipos en línea

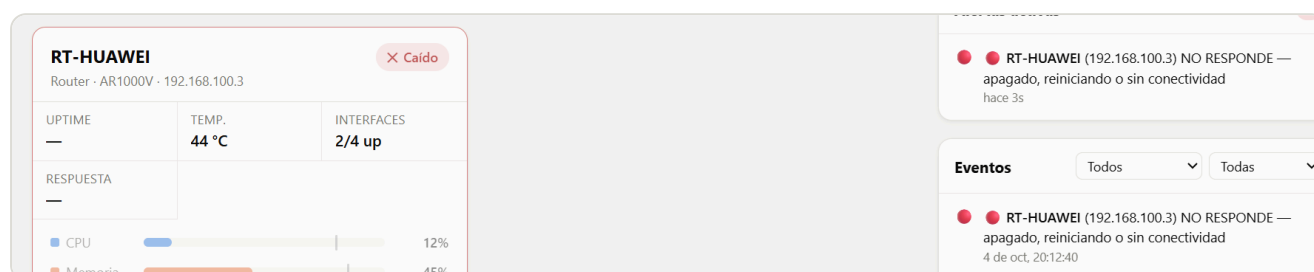
En `http://localhost:8090` el tablero muestra los tres equipos en **verde** («3/3 Todos respondiendo»), con su uptime, CPU, memoria, gráficas y la tabla de interfaces. Telegram recibe el mensaje «NetMon iniciado» y uno de «conectado» por cada equipo.



Captura 02. Tablero con los tres equipos en línea.

4 Simular una falla: equipo caído

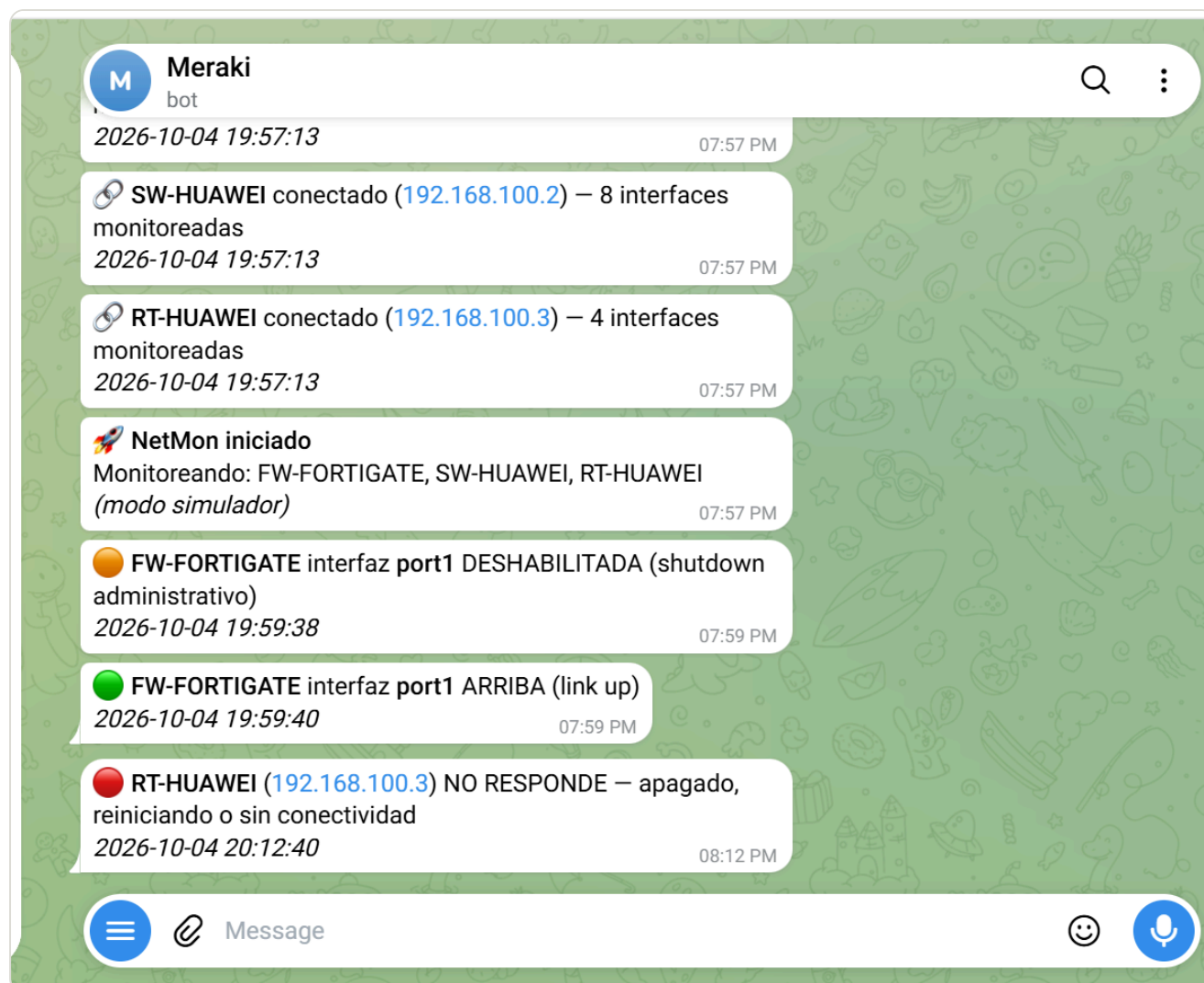
Desde el tablero se apagó el router RT-HUAWEI (botón «Apagar» del simulador, equivalente a desconectarlo de la red). Tras dos sondeos sin respuesta (unos 20 s), el tablero lo marcó en **rojo** como «Caído», con la alerta crítica en el panel de alertas activas.



Captura 03. Un equipo caído, marcado en rojo.

5 Recibir la alerta en Telegram

En el mismo momento (20:12:40) el bot envió la alerta «🔴 RT-HUAWEI (192.168.100.3) NO RESPONDE» al chat configurado. Antes habían llegado los avisos de inicio, de conexión de cada equipo y de una interfaz deshabilitada y recuperada (port1 del FortiGate).



Captura 04. Alerta recibida en Telegram.

5. Conclusiones

- **SNMP sigue siendo la base del monitoreo multimarca.** Con IF-MIB, que es estándar, se supervisan las interfaces de cualquier fabricante. Las MIB propias de Fortinet y Huawei solo se usan para CPU, memoria y temperatura, así que agregar otra marca es escribir un driver pequeño.
- **Combinar sondeo y traps da rapidez y confiabilidad.** El sondeo cada 10 s garantiza que ninguna falla pase desapercibida. Los traps reducen el tiempo de aviso a segundos cuando cae un enlace. Combinar SNMP con ping permite distinguir un equipo apagado de un error de configuración SNMP.
- **Docker hace el despliegue reproducible.** Con `docker compose up -d` la consola corre igual en cualquier PC. Cambiar de laboratorio o pasar a equipos de producción solo requiere editar las IPs en `config.yaml`.

- **Telegram acorta el tiempo de respuesta.** El responsable se entera de una caída en su celular sin tener el tablero abierto, y puede consultar el estado con `/estado` desde cualquier lugar.
- **Los secretos se separan del código.** Llevar el token y las communities a `.env`, excluido por `.gitignore`, permite publicar el repositorio sin exponer credenciales. Como mejora futura se propone migrar a SNMPv3 (autenticación y cifrado) y agregar más equipos y umbrales por equipo.